

Circuito RLC

Relatório referente a atividade do dia 11/11/2025.

Importar as bibliotecas necessárias:

- numpy para cálculos numéricos e vetores
- matplotlib.pyplot para gerar gráficos

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

Definir os parâmetros do circuito:

R: resistência

L: indutância

C: capacitância

Vx: tensão inicial no capacitor

```
3
4 # Parâmetros
5 R = 0.085
6 L = 11e-6
7 C = 180e-6
8 Vx = 900
```

Definir os valores iniciais das variáveis:

- i corrente inicial no indutor (0 A)
- vC tensão inicial no capacitor (vC= Vx)

```
9 # Condições iniciais
10 i = 0.0
11 vC = Vx
12
```

Definir os parâmetros de simulação no tempo:

t0 = tempo inicial

t_max = tempo final

dt = passo de integração

N = int(np.ceil(t_max / dt)) + 1

(faz o cálculo do número de pontos necessários para representar um intervalo de tempo.

- t_max = o tempo total de simulação (ou o tempo final)
- dt = o passo de tempo (incremento entre cada ponto)

Calcula quantos passos completos cabem no intervalo de tempo `np.ceil(...)` é usado porque, dependendo da divisão, você pode obter um número não inteiro, e precisa garantir que o número de pontos seja suficiente para cobrir todo o intervalo. Isso garante que você terá pontos suficientes até ultrapassar ou chegar no tempo final. `int(...)` converte o resultado para inteiro, já que o número de pontos deve ser inteiro. `+ 1` adiciona 1 porque, ao dividir o tempo, você calcula quantos intervalos existem, mas o número de pontos é sempre um a mais.)

```
13 # Tempo
14 t0 = 0.0
15 t_max = 200e-6 # 200 µs
16 dt = 0.02e-6 # 20 ns
17 N = int(np.ceil(t_max / dt)) + 1
18
```

Criar vetores preenchidos com zeros para armazenar:

- `t`: o tempo em cada passo
- `i_L`: corrente no indutor no tempo
- `v_C`: tensão no capacitor no tempo

```
# Vetores
t = np.zeros(N)
i_L = np.zeros(N)
v_C = np.zeros(N)
```

Armazena os valores iniciais na primeira posição de cada vetor (posição 0).

```
24 # inicializa índice 0 corretamente
25 t[0] = t0
26 i_L[0] = i
27 v_C[0] = vC
```

No final de cada repetição o código calcula o próximo estado.

Fazer uma análise no circuito para ver se o diodo está ou não em condução.

Diodo bloqueado

```
28
29 # Integração (Euler explícito)
30 for k in range(N-1):
31     # estado do diodo
32     if vC > 0:
33         di_dt = vC / L
34         dv_dt = - i / C
```

Diodo em condução

```

35     else:
36         di_dt = (vC - R * i) / L
37         dv_dt = - (i + vC / R) / C
38

```

Aplicar Método de Euler explícito: $y_{n+1} = y_n + h \cdot f(x_n, y_n)$

No código:

- di_dt e dv_dt = "Para onde estamos indo"
- i e vC = "Onde estamos agora"
- dt = "Quão longe olhamos no futuro"
- i_novo e vC_novo = "Onde estaremos"

Atualiza os valores para o próximo instante e armazena no vetor os valores recém-atualizados

$k+1$ sempre recebe o valor novo

```

39     # passo
40     i = i + di_dt * dt
41     vC = vC + dv_dt * dt
42
43     t[k+1] = t[k] + dt
44     i_L[k+1] = i
45     v_C[k+1] = vC

```

Índices dos extremos:

- $\text{argmax}()$ = posição do maior valor de i_L
- $\text{argmin}()$ = posição do menor v_C

Exibe o valor da corrente máxima e o instante em que ocorre

Exibe o valor mínimo da tensão e seu instante

Multiplica por $1e6$ para converter segundos em microssegundos

```

47     # resultados
48     idx_i_peak = np.argmax(i_L)
49     idx_v_min = np.argmin(v_C)
50     print(f"i_peak = {i_L[idx_i_peak]:.2f} A at {t[idx_i_peak]*1e6:.2f} μs")
51     print(f"v_min = {v_C[idx_v_min]:.2f} V at {t[idx_v_min]*1e6:.2f} μs")
52

```

Cria uma figura de 10 x 6 polegadas, divide a janela em 2 linhas × 1 coluna, seleciona o primeiro gráfico, plota corrente × tempo

'b' = cor azul

Tempo convertido para μs , desenha uma linha vertical onde ocorre o pico da corrente, define título, eixo e grade

```

53 # plots
54 plt.figure(figsize=(10,6))
55 plt.subplot(2,1,1)
56 plt.plot(t*1e6, i_L, 'b')
57 plt.axvline(t[idx_i_peak]*1e6, color='r', linestyle='--')
58 plt.title('Corrente no Indutor i(t)')
59 plt.ylabel('i (A)')
60 plt.grid(True)
61

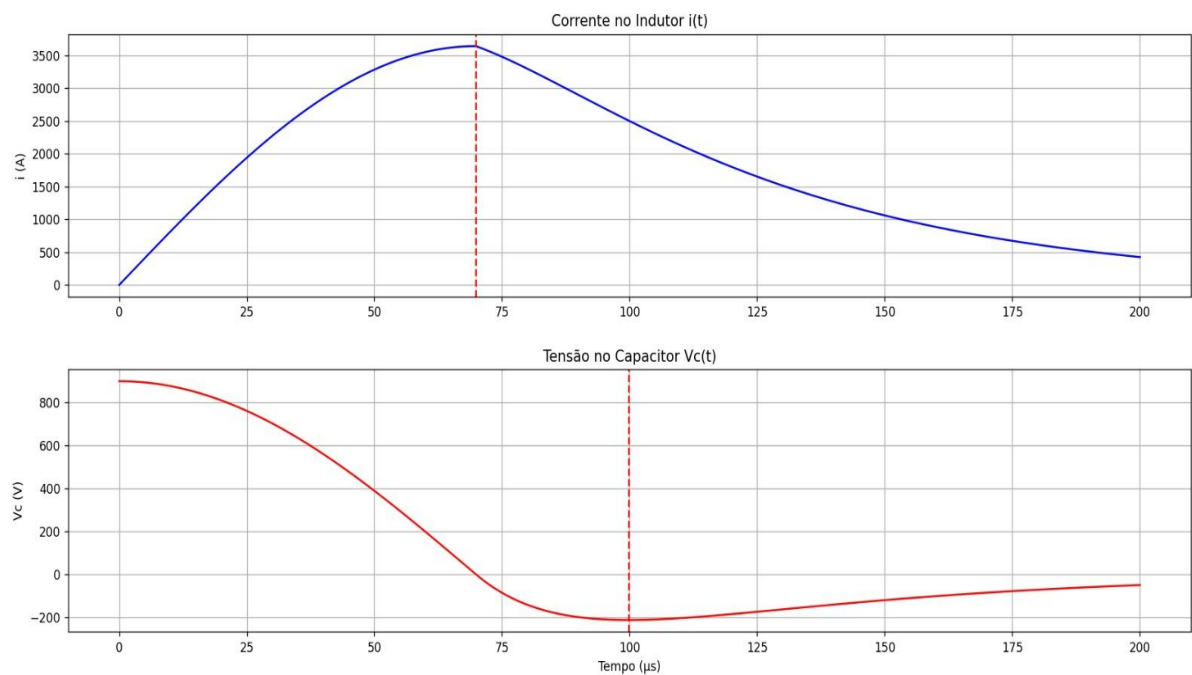
```

Seleciona o segundo gráfico, plota tensão e marca o valor mínimo, ajusta espaçamento e exibe os gráficos na tela

```

62 plt.subplot(2,1,2)
63 plt.plot(t*1e6, v_C, 'r')
64 plt.axvline(t[idx_v_min]*1e6, color='r', linestyle='--')
65 plt.title('Tensão no Capacitor Vc(t)')
66 plt.xlabel('Tempo (μs)')
67 plt.ylabel('Vc (V)')
68 plt.grid(True)
69 plt.tight_layout()
70 plt.show()

```



```

i_peak = 3641.96 A at 69.90 μs
v_min  = -212.91 V at 99.86 μs

```